Advanced Engineering Informatics 49 (2021) 101360

Contents lists available at ScienceDirect

Advanced Engineering Informatics

journal homepage: www.elsevier.com/locate/aei

Deep reinforcement learning-based safe interaction for industrial human-robot collaboration using intrinsic reward function

Quan Liu [a,b], Zhihao Liu [a,b,c], Bo Xiong [a,b], Wenjun Xu [a,b,*], Yang Liu [d,e,*]

[a] School of Information Engineering, Wuhan University of Technology, Wuhan 430070, China
[b] Hubei Key Laboratory of Broadband Wireless Communication and Sensor Networks (Wuhan University of Technology), Wuhan 430070, China
[c] Department of Production Engineering, KTH Royal Institute of Technology, SE-114 28 Stockholm, Sweden
[d] Department of Management and Engineering, Linköping University, SE-581 83 Linköping, Sweden
[e] Department of Production, University of Vaasa, 65200 Vaasa, Finland

ARTICLE INFO

Keywords:
Industrial human-robot collaboration
Collision avoidance
Deep reinforcement learning
Intrinsic reward function

ABSTRACT

Aiming at human-robot collaboration in manufacturing, the operator's safety is the primary issue during the manufacturing operations. This paper presents a deep reinforcement learning approach to realize the real-time collision-free motion planning of an industrial robot for human-robot collaboration. Firstly, the safe human-robot collaboration manufacturing problem is formulated into a Markov decision process, and the mathematical expression of the reward function design problem is given. The goal is that the robot can autonomously learn a policy to reduce the accumulated risk and assure the task completion time during human-robot collaboration. To transform our optimization object into a reward function to guide the robot to learn the expected behaviour, a reward function optimizing approach based on the deterministic policy gradient is proposed to learn a parameterized intrinsic reward function. The reward function for the agent to learn the policy is the sum of the intrinsic reward function and the extrinsic reward function. Then, a deep reinforcement learning algorithm intrinsic reward-deep deterministic policy gradient (IRDDPG), which is the combination of the DDPG algorithm and the reward function optimizing approach, is proposed to learn the expected collision avoidance policy. Finally, the proposed algorithm is tested in a simulation environment, and the results show that the industrial robot can learn the expected policy to achieve the safety assurance for industrial human-robot collaboration without missing the original target. Moreover, the reward function optimizing approach can help make up for the designed reward function and improve policy performance.

1. Introduction

Industrial robots are widely used in manufacturing. In some hybrid manufacturing scenarios, the safety fences in industrial robots workspace have been removed, and industrial robots need to assist humans in a shared workspace to complete manufacturing tasks [1]. Combining the flexibility of human and the efficiency of the industrial robot, human-robot collaboration (HRC) can improve productivity and quality of production in manufacturing while making the manufacturing process more flexible. However, the operator's safety is the primary issue during manufacturing operations, and industrial robots must perform allocated tasks with the assurance of the operator's safety. Therefore, industrial robots must be equipped with perceiving and decision-making capabilities to assure the safety of HRC applications. Fig. 1 shows a scenario of human-robot collaboration where safety needs to be considered.

Normally, a safe HRC manufacturing system can be implemented by a traditional motion planning algorithm based on visual sensors [2]. The vision sensors percept the distance between human and robot, and the safety of the collaborative process can be estimated through a risk evaluation method, which can guide the safe HRC manufacturing strategy. The motion planning algorithm based on the safety status and the human-robot distance generates a collision-free trajectory to ensure safety. However, collision avoidance for the robot has always been a challenge in robot motion planning, especially in an unstructured collaboration environment [3]. Traditional algorithms need to adjust

* Corresponding authors at: School of Information Engineering, Wuhan University of Technology, Wuhan 430070, China (W. Xu). Department of Management and Engineering, Linköping University, SE-581 83 Linköping, Sweden (Y. Liu).
E-mail addresses: quanliu@whut.edu.cn (Q. Liu), zhihaol@kth.se (Z. Liu), xiongbo@whut.edu.cn (B. Xiong), xuwenjun@whut.edu.cn (W. Xu), yang.liu@liu.se (Y. Liu).

https://doi.org/10.1016/j.aei.2021.101360
Received 18 November 2020; Received in revised form 18 March 2021; Accepted 7 July 2021
Available online 17 July 2021
1474-0346/© 2021 Elsevier Ltd. All rights reserved.

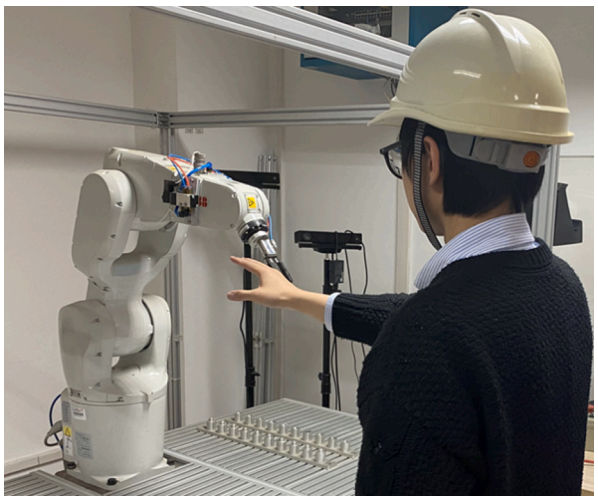


Fig. 1. A scenario of human-robot collaboration.

defined parameters to directly ensure performance, which is difficult to migrate to a new environment [4]. Therefore, it is necessary to propose an efficient and stable motion planning algorithm for industrial robots in HRC. Utilizing deep reinforcement learning (DRL) algorithms to solve the problem of robot safe control can significantly reduce human intervention in implementing a safe HRC manufacturing system to increase the robot's autonomous learning ability. Given a reward function, DRL algorithms can optimise control policy to maximize accumulated reward during interaction with the environment. However, one of the challenges in reinforcement learning (RL) problems is designing an appropriate reward function [5]. The difficulty lies in how to convert an optimization goal into a reward function correctly.

In general, it is relatively easy to obtain a fitness function related to the optimization problem based on our prior knowledge. The fitness function evaluates the performance of the policy, and it reflects the behaviour that we expect the robot to learn. However, the form of the fitness function is very rudimentary, and it also lacks guidance for the robot's behaviours. Therefore, results will not be charming even if the reward function is directly duplicated from the fitness function. Furthermore, for most problems, the reward function needs to be designed manually according to the fitness function or optimization goal and then guide the robot to learn. If the designed reward function has a clear property and definition, then the robot can learn satisfying performance through state-of-art DRL algorithms. Nevertheless, the design of the reward function becomes difficult and error-prone when the problem is complicated or contains multiple criteria. On our occasion, it is expected to reduce accumulated risk and assure task completion time. Due to the multi-object characteristic of our optimization problem, the reward value should be intuitively composed of weighted criteria, which leads to the problem of the allocation of weight contribution. Manually tuning these weights may significantly affect the learning of policy, causing unexpected behaviours of the robot, which results in the problem of reward-hacking [6]. Therefore, designing the reward function according to the fitness function is an iterative process, which requires a lot of trials and manual designs.

For the above reasons, a deterministic policy gradient [7] (DPG)-based reward function optimizing approach is proposed to reduce human intervention in reward function design. Concretely, a parameterized intrinsic reward function is introduced, and the reward function that acts over the interaction between the robot and environment is the combination of the manually designed reward function and the intrinsic reward function. The policy gradient updates the intrinsic reward function parameters to maximize the expected fitness so that the intrinsic reward function can be regarded as a supplement to the manually designed reward function. The manual adjustment process of the reward function design can be reduced by introducing the intrinsic reward function. This paper applies our method to the deep deterministic policy gradient (DDPG) [8] algorithm and evaluates it on a safe HRC manufacturing problem. The experiment results show that introducing a reward function optimizing approach in the DDPG algorithm can effectively improve the robot's policy performance. Moreover, even if the manually designed reward function has a deviation in weights contributions, the intrinsic reward function can modify it to ensure that the learned policy still has good performance over the fitness function.

The remainder of this paper is organized as follows. In Section 2, the related work about safe HRC manufacturing system and DRL in robot control and reward function design in DRL is introduced and discussed. Section 3 describes and formulates the background of DRL, the safe HRC problem and the reward design problem. In Section 4, the DRL algorithm intrinsic reward-deep deterministic policy gradient (IRDDPG) is described in detail. Section 5 performs the experiments to verify and analyze the proposed algorithm. Finally, Section 6 concludes the proposed research.

2. Related work

Safe human-robot collaboration lies in the frontline of robotics research. Among methodologies enhancing human-robot collaboration security, controlling the robot is crucial in the whole robotic system. This section first introduced the safe human-robot collaboration towards manufacturing tasks, following with the reviews on the deep reinforcement learning technology and its reward function design issue.

2.1. Safe human-robot collaborative manufacturing system

In recent years, the safe HRC as the top issue in the industrial collaborative environment has been widely investigated [9]. The industrial robot safety standard ISO 10,218 [10] defines four collaboration modes: safety level monitoring stop, manual guidance, speed and separation monitoring (SSM), power and force limits, and the collaborative robot safety specification ISO/TS 15066 [11] complements the detailed concepts and requirements. The collaboration mode SSM has been widely applied to HRC scenarios due to its flexible and relatively easy to implement features [12]. As a specified implementation of SSM, the solutions of collision avoidance of industrial robots have been studied by many researchers in safe HRC manufacturing. The related research work can be divided into three aspects: status perception of HRC process, risk assessment and safety-related strategy of industrial robots [13].

Using appropriate external sensors to real-time monitor obstacles and human in the collaborative workspace is a prerequisite for a safe HRC system [14]. Flacco et al. [15] proposed a method for calculating the minimum distance between robot joints and moving obstacles in-depth space, and this method can achieve fast calculating frequency to ensure real-time performance. Mohammed et al. [16] utilized multiple depth cameras to detect collision in the HRC environment in real-time and proposed a prototype system to control the industrial robot based on the collision detection result, thereby achieving active collision avoidance. Ragaglia et al. [17] proposed a trajectory generation algorithm for safe HRC, which fused the data of multiple depth sensors and modified the pre-programmed trajectory to avoid a collision. Risk assessment combines the collision detection result, state of industrial robots, task types and equipment errors to evaluate the danger of HRC scenario. From a task-based perspective, Marvel et al. [18] proposed a risk assessment method to offline evaluate the safety of the HRC system at the beginning of the task. Based on the running trajectory of the industrial robot, Vicentini et al. [19] proposed a dynamic SSM method to establish a minimum safe area. The size of the safe area changes with the speed of the industrial robot, and the dynamic change of the safe area makes HRC manufacturing more flexible. Polverini et al. [20] proposed a method to calculate the hazard value at a certain point in the robot's workspace. This method first calculates the hazard value generated at a

certain point on the robot's joints, and then the danger filed of the collaborative workspace can be obtained by surface integration. Based on the linear relationship on the robot link speed, Zanchettin et al. [21] derived a set of inequalities to determine whether a certain point in the collaborative workspace is safe and visualized the danger filed generated by the robot. Rubagotti et al. [22] investigated a predictive model for obstacle-avoidance robot teleoperation.

Risk assessment can calculate the risk value in the current state of HRC in real-time and guide robot motion planning to achieve collision avoidance. Collision avoidance based robot motion planning can generate a collision-free trajectory that can reach the target point, and many researchers have investigated solution on collision avoidance in the field of safe HRC [12]. For the unstructured collaboration environment, industrial robots reactive control has been widely used in safe HRC, which holds the idea that virtual repulsive forces or impedances generated by obstacles will drive the robot away from obstacles [23]. Lo et al. [24] used virtual impedance control operating in the risk space to achieve collision avoidance for redundant robots, where the risk space is a vector space describing all possible collisions. This method regulates collision avoidance dynamics as a second-order linear response in the risk space and can generate a smooth and continuous trajectory. Based on the speed of the robot end effector (EE) and the relative distance between the EE and the obstacle, Schmidt et al. [25] proposed a robot active response control method to change the running speed and direction of the robot in real-time. However, this method only considers the collision between the EE of the robot and the human. Kardan et al. [26] proposed a collision avoidance method based on real-time speed scaling, which combines speed scaling with virtual impedances generated by obstacles to achieve real-time collision avoidance.

2.2. Deep reinforcement learning in robot control

The traditional robot motion control algorithms need to manually adjust the algorithm parameters and consider all possible situations in the collaborative environment. However, DRL algorithms can enable the robot to learn the control policy autonomously with the minimal human intervention [27] and increase the robot's intelligence and autonomy to adapt to the changes and rapid development of robot applications. In recent years, researchers have done much research on DRL-based robot control. In general, the robot control problem can be formulated into a sequential decision problem, while RL algorithms have a strong capability in solving this kind of sequential problem [28]. With the rapid development of deep learning, the combination of deep neural networks and RL algorithms can enable RL algorithms to solve high-dimensional and complex problems. Mnih et al. [29] successfully combined the RL algorithm with deep learning techniques through the freezing technique and the experience replay technique, where deep neural networks acted as the nonlinear parameters approximator of Q-table, making the agent able to learn policy in continuous input. Using CNN to extract state feature from the raw image, deep Q-network (DQN) in [29] implemented mapping from the raw image to agent action and achieved great success in video games. Zhang et al. [30] utilized DQN to realize the target-reaching task of a three-degree freedoms manipulator. Sangiovanni et al. [31] improved robotic motion planning through deep reinforcement learning and tested their method in the V-REP simulator.

Although DQN achieved great success, it is not feasible to apply DQN to RL problem with continuous action space such as high degree freedoms manipulator control [8]. Gu et al. [32] proposed the Normalized Advantage Function (NAF) algorithm to realize robot control in continuous action space. NAF algorithm is a continuous action version of the Q-learning algorithm, and it explores through the learned model to accelerate the learning speed of model-free RL algorithm, thereby improving the sampling efficiency. Sangiovanni et al. [33] proposed a NAF-based method to achieve collision avoidance in HRC. However, it only considered the collision of human and the EE of a robot rather than the entire robot. Besides, the DDPG [8] algorithm is also widely used in robot control. By utilizing two 2D views of the environment rather than a depth image, Paul et al. [34] proposed a DDPG-based framework to implement the path planning for the random target reach. Do et al. [35] presented an approach to learn to pour water for a robot using DDPG and successfully transferred the learned policy in simulation to the real robot.

2.3. Reward function design in deep reinforcement learning

Although DRL algorithms have achieved great success in robot control, the premise of many algorithms is that the reward function is known. However, for most problems, researchers need to design the appropriate reward function manually and make the robot capture the designer's expected behaviour through the reward function [5], which arises the reward function design problem. Therefore, for this problem that requires the reward function to express the desired behaviour correctly, one way to solve the problem is to add an additional reward to the existing reward function to achieve a certain kind of transformation of the existing reward function. By transforming the reward function, the robot can obtain more suitable samples for learning. The author of the literature [36] proved that the optimal control policy would not change when the added reward value is potential-based. However, due to deep neural network fitting, robot exploration, and computing resources limitations, it is often difficult for robots to learn an optimal policy for many complex reinforcement learning problems. Therefore, even if the added reward value is not potential-based, the robot can still learn a good control policy. Therefore, auxiliary rewards or curiosity-driven rewards is often used as a reward function transformation to help robots learn, e.g., the count-based reward value is used as an auxiliary reward to help the robot explore the state of less visit [37,38]. Pathak et al. Paper [39] proposed an intrinsic curiosity module to generate additional reward values to help robots explore.

For some complex problems, such as a dexterous grasping task for a manipulator, it is challenging and error-prone to design the correct reward function manually. Thus some researchers have investigated how to use expert demonstrations to help robots learn, which mainly includes imitation learning [40] and inverse reinforcement learning (IRL) [41]. IRL algorithms learn the reward function related to the task through expert demonstration and then learn the control policy according to the learned reward function. The idea of imitation learning is similar to supervised learning, which minimises the error between policy output and expert demonstrations. Pfeiffer et al. [42] applied imitation learning to mobile robot motion planning, and the robot learned policy from expert data generated by traditional motion planning algorithm.

To solve the reward function design problem, some researchers have proposed the optimal reward framework [43,44]. The research work related to the reward function design in this paper is also based on this framework. Research work on the optimal reward framework shows that the reward function defined directly by the optimization goal does not guarantee that the robot can learn the optimal policy. Based on the optimal reward framework, Sorg et al. [45] proposed a reward function design method based on stochastic policy gradient, which enabled the robot to obtain the optimal reward function through learning instead of exhaustive searching. On this basis, Zheng et al. [46] proposed a learning method of intrinsic reward function based on policy gradient. This method enables the robot to learn the intrinsic reward function while learning the control policy, thereby promoting the robot to learn better control policy.

From the related publication, we can learn that the safety issue is the top priority in human-robot collaboration, especially in manufacturing tasks. In the perspective of artificial intelligence, deep reinforcement learning has shown a strong capability to enrich an agent's intelligence in a complex task. However, the design of the reward function is so tricky that it should be designed carefully towards the given tasks. Investigating deep reinforcement learning in safe human-robot

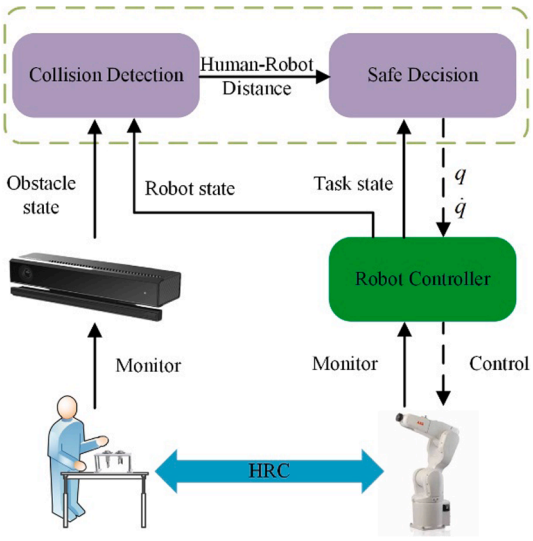


Fig. 2. The framework of collision avoidance in safe HRC.

collaboration needs to be further studied.

3. Problem definition

In this chapter, we introduced the background of DRL and defined the main two problems of this research mathematically given the industrial human-robot collaboration scenario. Respectively, the safe human-robot collaboration problem with manufacturing tasks is defined with the Markov decision process and the specifically designed reward function.

3.1. Deep reinforcement learning

Deep reinforcement learning is an extension of reinforcement learning using deep neural networks to represent the value functions. In reinforcement learning, the term "agent" is used for the role of a decision-maker to interact with the environment. Markov decision process is utilized to illustrate this sequential decision problem. It can be defined by a tuple $< S, A, P, R, T, \gamma >$, which consists of a state set S, an action set A, a state transition function $P(s_{t+1}|s_t, a_t)$, a reward function $R(s_t, a_t)$, the finite planning horizon T and a discount factor γ for calculating the expected discounted return $\mathbb{E}\left[\sum_{i=0}^{T-1}\gamma^t r_t\right]$. Details about the reinforcement learning theory can be found in [47].

3.2. Safe human-robot collaborative manufacturing problem

This section considers the scenario that one human and one industrial robot perform collaborative manufacturing in a shared workspace. For a given manufacturing task, the industrial robot must not collide with a human during the process of moving to the target configuration. On the other hand, human activity in the workspace can affect the movement of the industrial robot. As a result, the industrial robot needs to perceive the human position and adjust the trajectory in real-time.

In general, the collision avoidance process can be divided into collision detection and safe decision. The framework for handling collision avoidance in safe HRC is illustrated in Fig. 2. For collision detection, the human position is perceived by a depth camera and then the minimum distance between the human and the industrial robot can be calculated. Based on the human-robot distance and task state, safe decision generates robot commands to control the robot to ensure human safety and productivity.

Considering that human and industrial robot cannot be regarded as particles in HRC during collision detection, the bounding box technology is used. In this work, capsules bounding boxes [48] are utilized to model the industrial robot and human. Considering that the human arm is the main obstacle in the collaborative workspace and the joint information outside the workspace can be ignored, this paper only considers the right hand and right elbow of human during collision detection. The collision bounding boxes for HRC are illustrated in Fig. 3, where the industrial robot's links are modelled as four capsules, and the human arm is modelled as a capsule. The joints' spatial coordinates of the industrial robot can be computed by the kinematics model, and the skeleton data of human can be perceived by Kinect V2. On this basis, the spatial representation of the collision bounding boxes can be determined according to the radius of each capsule bounding box.

Then we constructed the collision detection model in HRC to calculate the human-robot distance, and the collision detection in HRC is shown in Fig. 4.

The distance between the industrial robot and human can be computed based on the collision detection model, as shown in Fig. 4(c), where only $||d_{1,t}||$and$||d_{2,t}||$ are drawn for better visual effects. Therefore,

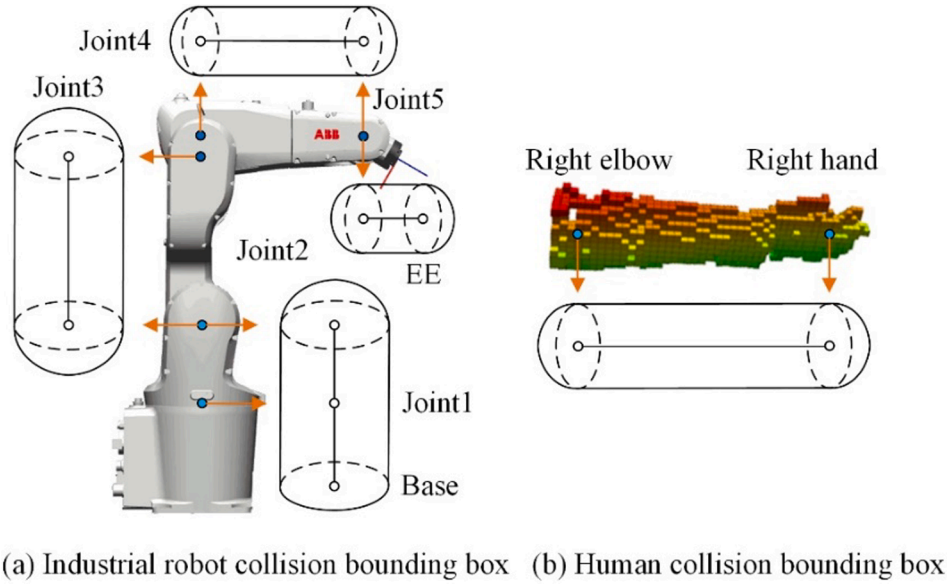


Fig. 3. Illustration of the collision bounding boxes for HRC.

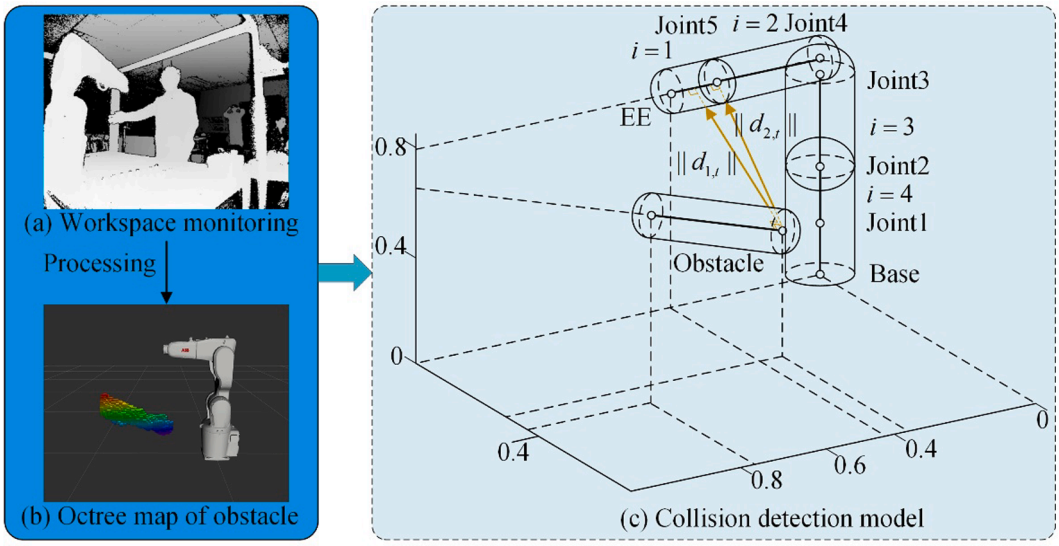


Fig. 4. Illustration of collision detection for HRC.

we can define the distance between the industrial robot and human:

Definition 1. (*Distance between the industrial robot and human*) The distance between the industrial robot and human is defined as a vector $D(c_t, o_t) = \left(||d_{1,t}||, ||d_{2,t}||, ||d_{3,t}||, ||d_{4,t}||\right)$ at time step t, where c_t is the industrial robot configuration and o_t is the human position. The distance norm $||d_{i,t}||$ is the minimum distance between the obstacle capsule and i-th link capsule at time step t, where $i = 1, 2, 3, 4$.

When the distance between the industrial robot and human is less than the safe distance, the industrial robot needs to adjust the trajectory to complete the manufacturing task while avoiding collision with the human. Based on the calculated distance $D(c_t, o_t)$, we adopt the finite Markov Decision Process (MDP) to formalize a safe HRC manufacturing problem. For our problem, at each time step t, the system state $s_t \in S$ is a vector $\left(c_t, c_{target}, o_t\right)$, which is the concatenation of the robot state c_t, target state c_{target}which represents the joint angle for completing the task, and obstacle state o_t representing the human position. The action $a_t \in A$ that the agent takes is the velocity for each joint. Based on the transition function $P(s_{t+1}|s_t, a_t)$, the agent reaches a new state s_{t+1} and gets a reward $r_t = R(s_t, a_t)$ given from the environment.

The reward function reflects the optimization goal that the industrial robot should adjust the trajectory to reduce accumulated risk and assure task completion time. Then the safe HRC manufacturing problem can be defined:

Problem 1. (*Safe HRC manufacturing problem*) Given the initial and target task state $c_{initial}, c_{target} \in C_{space}$, a reach tolerance ε, and time interval Δt, the goal of safe HRC manufacturing problem is to find a policy $\pi : s_t \rightarrow a_t$ maximizes the expected discounted return when moving from $c_{initial}$ to c_{target}:

$$\pi^* = \underset{\pi}{argmax}\mathbb{E}_\pi\left[\sum_{t=0}^{T-1}\gamma^t r_t | s_0\right] \tag{1}$$

For each state s_t, the minimal safety constrains is $D(c_t, o_t) > 0$, and the state transition is $c_{t+1} = c_t + \pi(s_t)\cdot\Delta t$. The task is completed when the robot state satisfies the condition: $\|c_t - c_{target}\| \leq \varepsilon$.

Concretely, the DRL algorithm is used to solve the safe HRC manufacturing problem, and the deep neural network is used to parameterize the industrial robot's control policy, which can be regarded as the nonlinear approximator of collision detection and safety decision.

3.3. Reward function design problem

Given a specific reward function, the RL agent can learn a policy to maximize the expected discounted return [5]. Therefore, the key point of the safe HRC manufacturing problem is how to design an appropriate reward function following our optimization goal. In general, for the multi-object optimization problem, the designed reward function contains parts corresponding to each optimization goal. The total reward value is obtained in the form of a weighted sum, e.g. for the safe HRC manufacturing problem, the reward function can be designed in the form as in equation:

$$R(s_t, a_t) = \lambda_1 \cdot R_{risk}(s_t, a_t) + \lambda_2 \cdot R_{task}(s_t, a_t) \tag{2}$$

The allocation of weight contribution makes a great impact on policy learning, which arises the reward design problem.

Therefore, we build our work on the Optimal Reward Framework proposed by Singh et al. [43,44]. The optimal reward framework mainly contains three concepts: a bounded RL agent that has limited computation resource, a fitness function $R^{fitness}$ which means the designer's preference or goal over agent policy π, and space of reward functions $\mathcal{R}$. Based on the most intuitive understanding of our optimization goal, we can define the fitness function related to accumulated risk and expected task completion time:

Definition 2. (*Fitness function of safe HRC manufacturing problem*) In this research, the robot is expected to conduct aimed task while assuring safety in the collaboration. Hence, indicators about task and risk need to be considered simultaneously. It is solved by a comprehensive fitness function containing two parts: risk fitness $Risk^{fitness}$ and task fitness $Task^{fitness}$. For the risk fitness $Risk^{fitness}$, at time step t, for each $||d_{i,t}||$ in $D(c_t, o_t)$, if $||d_{i,t}|| < d_{safe}$, the risk fitness is decreased by one, and if $||d_{i,t}|| <= 0$, the risk fitness is decreased by five, where is the safe distance. For the task fitness $Task^{fitness}$, if the agent completes the task, the task fitness is incremented by ten, and if the task completion time is over $T_{expected}$, the task fitness is decreased by ten, where $T_{expected}$ is the expected task completion time. The total fitness for one interaction is the sum of risk fitness and task fitness: $R^{fitness} = Risk^{fitness} + Task^{fitness}$.

The risk evaluation is based on the distance $D(c_t, o_t)$ and safe distance d_{safe}, and if the robot enters an unsafe state, that is $||d_{i,t}|| < d_{safe}$ or $||d_{i,t}|| <= 0$, the robot will get a penalty. Only when the robot completes the task will there be a positive reward value. When the completion time exceeds the expected task completion time, a penalty will also be added

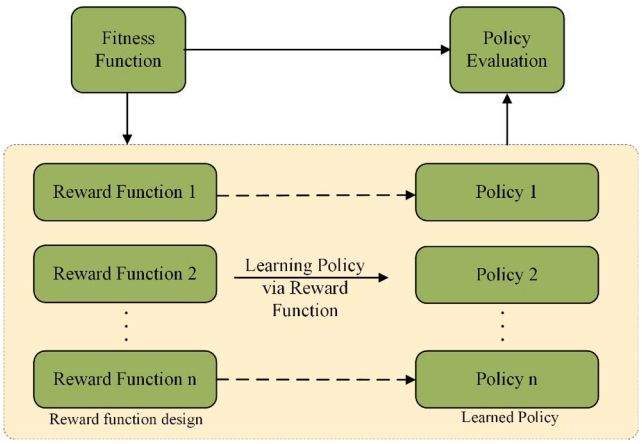


Fig. 5. The principle of the reward design problem.

Table 1
Main symbol and definition.

Symbol	Definition
$\theta^{in}, \theta^{\mu}$	Parameters of intrinsic reward function and policy
$r_t^{fitness} = R^{fitness}(s_t, a_t)$	Fitness at time step t
$r_t^{ex} = R^{ex}(s_t, a_t)$	Extrinsic reward at time step t
$r_t^{in} = R_{\theta^{in}}^{in}(s_t, a_t)$	Intrinsic reward at time step t
$r_t = r_t^{in} + r_t^{ex}$	Total reward for the learning of policy
$Q^{in}(s_t, a_t) = \mathbb{E}_{s_t \sim Env}\left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k}^{in} \middle\| s_t, a_t\right]$	Intrinsic action-value function
$Q^{ex}(s_t, a_t) = \mathbb{E}_{s_t \sim Env}\left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k}^{ex} \middle\| s_t, a_t\right]$	Extrinsic action-value function
$Q(s_t, a_t) = \mathbb{E}_{s_t \sim Env}\left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k} \middle\| s_t, a_t\right] = Q^{in}(s_t, a_t) + Q^{ex}(s_t, a_t)$	Action-value function
$Q^{fitness}(s_t, a_t) = \mathbb{E}_{s_t \sim Env}\left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k}^{fitness} \middle\| s_t, a_t\right]$	Action-value function based on fitness
$J^{fitness}(\mu) = \mathbb{E}_{s_t \sim Env}\left[\sum_{t=0}^{T-1} \gamma^t r_t^{fitness} \middle\| s_0\right]$	Policy performance on a fitness function
$J(\mu) = \mathbb{E}_{s_t \sim Env}\left[\sum_{t=0}^{T-1} \gamma^t r_t \middle\| s_0\right]$	Policy performance on total reward

to the total fitness.

The fitness function produces the expected accumulated fitness $\mathbb{E}_{\pi}\left[\sum_{t=0}^{T-1} \gamma^k r_t^{fitness}\right]$ as a scalar to evaluate the policy π. The agent's policy varies with different choices of the reward function, which then affects the value of the expected accumulated fitness. Fig. 5 shows the principle of the reward design problem. Then we can formulate the reward design problem based on the safe HRC manufacturing problem:

Problem 2. (*Reward design problem*) The reward design problem is defined as a tuple $< S, A, P, R^{fitness}, T, \gamma, \mathcal{R} >$, the reward design problem is to design a reward function $R \in \mathcal{R}$ for learning a policy that can maximize the expected accumulated fitness, which can be expressed by the following equation:

$$R^* = \underset{R \in \mathcal{R}}{argmax} \mathbb{E}_{\pi}\left[\sum_{t=0}^{T-1} \gamma^k r_t^{fitness} | \pi(\cdot|R)\right] \tag{3}$$

where $\pi(\cdot|R)$ is the learned policy given a reward function R, and here we omit the representation of the environment's distribution.

Combining the safe HRC manufacturing problem and the reward function design problem, it can be stated that the appropriate reward function needs to be designed according to the fitness function, and then the robot can learn the safe control policy through the DRL algorithm. Besides, the fitness function should be in the space of reward functions $\mathcal{R}$, that is $R^{fitness} \in \mathcal{R}$, so the fitness function can be selected as the reward function.

4. Reward function optimizing based DDPG learning method

For the reward design problem, the choice of the reward function will affect the learning of policy and further affect the expected fitness value. Therefore, the designed reward function decides the expected fitness. Previous work [43] used the exhaustive search over the reward functions space to find the optimal reward function, which will cost much time on a complicated problem. Zheng et al. [46] proposed an algorithm for the agent learning intrinsic reward function while learning the policy.

Based on the above idea, we proposed an online reward function optimizing approach to help solve the reward design problem. To be more specific, the reward function for the agent to learn a policy is the sum of the extrinsic reward function and the intrinsic reward function. The extrinsic reward function is designed manually based on our prior knowledge, and the intrinsic reward function is a parameterized function. The parameters of the intrinsic reward function are updated by the policy gradient to maximize the expected accumulated fitness. As discussed earlier, the manual design of the reward function is difficult and error-prone when the problem is complicated or contains multiple criteria. Therefore, introducing the intrinsic reward function optimizes the manually designed reward function to improve the expected accumulated fitness.

4.1. DPG-based reward function optimizing approach

Firstly, we briefly discuss the DPG [7] algorithm, the deterministic version of the policy gradient algorithm, and then we give the DPG-based reward function optimizing approach.

Policy gradient-based algorithms are widely used in RL problems with continuous action space. For the deterministic and stochastic version of policy gradient, the DPG can be estimated more efficiently than the stochastic policy gradient. For the RL problem with the deterministic policy $\mu(s|\theta^{\mu})$, the performance of policy can be denoted by the expected discounted return $J(\mu) = \mathbb{E}_{s_t \sim Env}\left[\sum_{t=0}^{T-1} \gamma^t r_t \middle\| s_0\right]$, where the state is chosen from state distribution in the environment. The DPG algorithm proved that the gradient of the deterministic policy's performance could be obtained by applying the chain rule to the expected discounted return concerning the policy parameters θ^{μ}:

$$\nabla_{\theta^{\mu}} J(\mu) = \mathbb{E}_{s_t \sim \rho^{\beta}}\left[\nabla_a Q(s_t, a_t)|_{a_t = \mu(s_t)} \nabla_{\theta^{\mu}} \mu(s_t|\theta^{\mu})\right] \tag{4}$$

$Q(s_t, a_t) = \mathbb{E}_{s_t \sim Env}\left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k} \middle\| s_t, a_t\right]$ is the action-value function corresponding to policy $\mu(s|\theta^{\mu})$.

Then based on the DPG theorem, we proposed the DPG-based reward function optimizing approach. The proposed approach introduced the intrinsic reward function to generate intrinsic reward to help the robot learn a policy. Unlike [46], the intrinsic reward function parameters are updated by the expected accumulated fitness rather than the expected extrinsic return. Therefore, the intrinsic reward function can optimize the total reward to improve the expected accumulated fitness. At first, we define the notations used in Table 1:

Updating the parameters θ^{μ} in the direction of policy gradient $\nabla_{\theta^{\mu}} J(\mu)$ can improve the performance of policy $\mu(s|\theta^{\mu})$. Therefore, the idea of our approach is that the updates of intrinsic reward function parameters θ^{in} can make the total reward r_t change in the direction of improving $J^{fitness}(\mu)$, and then make the policy parameters θ^{μ} update in the direction of improving $J^{fitness}(\mu)$.

Similar to the policy gradient algorithm, the most straightforward way to update the intrinsic reward function is to calculate $\nabla_{\theta^{in}} J^{fitness}(\mu)$, the gradient of the policy performance on the fitness function $J^{fitness}(\mu)$ concerning intrinsic reward function parameters θ^{in}. Considering that the changes of θ^{in} can affect policy $\mu(s|\theta^{\mu})$ and then affect $J^{fitness}(\mu)$, so the

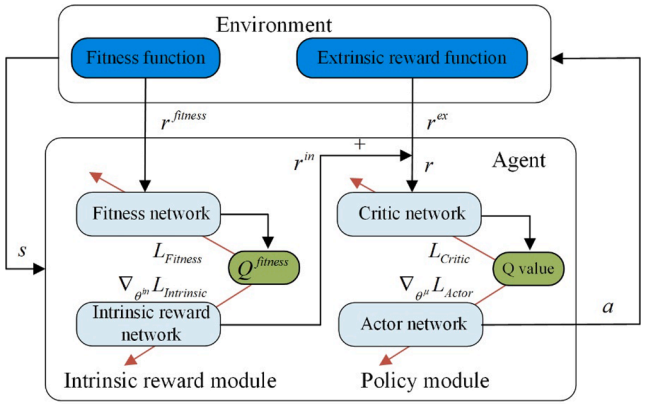


Fig. 6. IRDDPG learning architecture.

gradient $\nabla_{\theta^{in}} J^{fitness}(\mu)$ can be divided into two gradients [45,46]:

$$\nabla_{\theta^{in}} J^{fitness}(\mu) = \nabla_{\theta^{\mu}} J^{fitness}(\mu) \nabla_{\theta^{in}} \theta^{\mu} \tag{5}$$

The first gradient $\nabla_{\theta^{\mu}} J^{fitness}(\mu)$ is the policy gradient on the fitness function, which can be computed as

$$\nabla_{\theta^{\mu}} J^{fitness}(\mu) = \mathbb{E}_{s_t \sim \rho^{\beta}} \left[\nabla_a Q^{fitness}(s_t, a_t)|_{a_t=\mu(s_t)} \nabla_{\theta^{\mu}} \mu(s_t|\theta^{\mu}) \right] \tag{6}$$

and the second gradient $\nabla_{\theta^{in}} \theta^{\mu}$ is the gradient of the policy concerning the intrinsic reward function parameters. The update of policy parameters θ^{μ} can be obtained by policy gradient $\nabla_{\theta^{\mu}} J(\mu)$:

$$\theta^{\mu} \approx \theta^{\mu'} + \alpha \nabla_{\theta^{\mu'}} J(\mu) \tag{7}$$

Then the second gradient can be computed as:

$$\begin{aligned}
\nabla_{\theta^{in}} \theta^{\mu} &= \nabla_{\theta^{in}} (\theta^{\mu'} + \alpha \nabla_{\theta^{\mu'}} J(\mu)) \\
&= \alpha \nabla_{\theta^{in}} (\nabla_{\theta^{\mu'}} J(\mu)) \\
&= \alpha \nabla_{\theta^{in}} \left(\mathbb{E}_{s_t \sim \rho^{\beta}} \left[\nabla_a Q(s_t, a_t)|_{a_t=\mu(s_t)} \nabla_{\theta^{\mu'}} \mu(s_t|\theta^{\mu'}) \right] \right) \\
&= \alpha \nabla_{\theta^{in}} \left(\mathbb{E}_{s_t \sim \rho^{\beta}} \left[\nabla_a Q^{in}(s_t, a_t)|_{a_t=\mu(s_t)} \nabla_{\theta^{\mu'}} \mu(s_t|\theta^{\mu'}) \right] \right) \\
&= \alpha \nabla_{\theta^{in}} \left(\mathbb{E}_{s_t \sim \rho^{\beta}} \left[\nabla_a \left(\mathbb{E}_{s_t \ Env} \left[\sum_{k=0}^{T-t-1} \gamma^k r_{t+k}^{in} \middle| s_t, a_t \right] |_{a_t=\mu(s_t)} \nabla_{\theta^{\mu'}} \mu(s_t|\theta^{\mu'}) \right] \right) \right)
\end{aligned} \tag{8}$$

The calculation of the first and the second gradient uses different policy parameters, so the calculation needs to be kept in a particular order. At first, $\nabla_{\theta^{in}} \theta^{\mu}$ is computed by the policy parameters $\theta^{\mu'}$, and then the policy is updated by $\nabla_{\theta^{\mu}} J(\mu)$. Then $\nabla_{\theta^{\mu}} J^{fitness}(\mu)$ can be computed by the updated policy parameters θ^{μ}. Finally, the intrinsic reward function parameters θ^{in} can be updated by applying the product of $\nabla_{\theta^{\mu}} J^{fitness}(\mu)$ and $\nabla_{\theta^{in}} \theta^{\mu}$. Note that the DPG removes the integral over action, so the importance of sampling can be avoided, although off-policy updates are applied to obtain samples and calculate gradients.

4.2. Intrinsic reward-DDPG learning algorithm

Combined with the deep learning techniques and DPG algorithm, the DDPG algorithm can learn promising policy on robot control problem with continuous state and action space. We apply our DPG-based reward function optimizing approach to the DDPG algorithm and propose the intrinsic reward DDPG (IRDDPG) algorithm to learn the safe control policy. The IRDDPG learning architecture is shown in Fig. 6. The agent contains two modules, a policy module built based on the DDPG algorithm and an intrinsic reward module built according to our approach.

The intrinsic reward module maintains a fitness network $Q^{fitness}(s_t, a_t|\theta^{fitness})$ which estimates the action-value based on fitness and an intrinsic reward network$R_{\theta^{in}}^{in}(s_t, a_t)$ which outputs the intrinsic reward for each state. For the policy module, the critic network $Q(s_t, a_t|\theta^{Q})$ estimates the action-value based on the total reward, and the actor-network $\mu(s_t|\theta^{\mu})$ maps a state to a deterministic action. During the interaction with the environment, the agent executes the action a_t from the actor-network according to state s_t, and the environment feeds back the fitness $r_t^{fitness}$ and the extrinsic reward r_t^{ex} to the agent. The fitness $r_t^{fitness}$ acts on the fitness network.

Based on the Bellman equation, the loss for updating the critic networks can be defined as:

$$L_{Critic} = \frac{1}{N} \sum_{i=0}^{N-1} \left(y_i - Q(s_i, a_i|\theta^{Q}) \right)^2 \tag{9}$$

where $y_i = r_i + \gamma Q'(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q'})$. $\theta^{\mu'}$ and $\theta^{Q'}$ are the parameters of the target actor and critic network, respectively. The actor-network parameters can be updated by applying the policy gradient:

$$\nabla_{\theta^{\mu}} L_{Actor} \approx \frac{1}{N} \sum_{i=0}^{N-1} \nabla_a Q(s_i, a_i|\theta^{Q})|_{a_i=\mu(s_i)} \nabla_{\theta^{\mu}} \mu(s_i|\theta^{\mu}) \tag{10}$$

The update of the fitness network is analogous to the critic network, which can be defined as:

$$L_{Fitness} = \frac{1}{N} \sum_{i=0}^{N-1} \left(y_i^{fitness} - Q^{fitness}(s_i, a_i|\theta^{Q^{fitness}}) \right)^2 \tag{11}$$

where $y_i^{fitness} = r_i^{fitness} + \gamma Q^{fitness'}(s_{i+1}, \mu'(s_{i+1}|\theta^{\mu'})|\theta^{Q^{fitness'}})$, and $\theta^{Q^{fitness'}}$ is the parameters of the target fitness network. The parameters of the intrinsic reward network can be updated by Equation 4–2:

$$\nabla_{\theta^{in}} L_{Intinsic} = \nabla_{\theta^{in}} J^{fitness}(\mu) = \nabla_{\theta^{\mu}} J^{fitness}(\mu) \nabla_{\theta^{in}} \theta^{\mu} \tag{12}$$

The IRDDPG algorithm is presented in Algorithm 1 in detail. Except for the intrinsic reward network, target networks that copy the other networks are used to calculate target action-value estimation and target policy. The parameters of target networks are updated by soft parameters replacement to improve the stability of learning. A Gaussian process is used to add noise to action to help exploration. After one interaction with the environment, the agent is trained for certain steps. Algorithm 1 includes the interaction process between the agent and the environment (lines 5–11), and the training process for the agent (lines 12–18).

Algorithm 1 IRDDPG learning algorithm

1. **Inputs:** Random initialize critic network parameters θ^{Q}, target critic network parameters $\theta^{Q'}$, actor-network parameters θ^{μ}, target actor-network parameters $\theta^{\mu'}$; fitness network parameters $\theta^{Q^{fitness}}$, target fitness network parameters $\theta^{Q^{fitness'}}$; intrinsic reward network parameters θ^{in}.
2. **Inputs:** Action noise $\mathcal{N}$; Target network replacement factor τ; Environment *Env*; Initialize replay buffer $\mathcal{D}^{replay}$.
3. **Outputs:** Actor-network $\mu(s_t|\theta^{\mu})$.
4. **For** episode $e \in \{1, 2, \ldots\}$ **do**
5. Initialize state s_0 from *Env*
6. **For** $t \in \{0, \ldots, T-1\}$ **do**
7. Sample a noise n_t from $\mathcal{N}$
8. Choose an action$a_t = \mu(s_t|\theta^{\mu}) + n_t$
9. Execute action a_t and observe the new state s_{t+1} and get extrinsic reward r_t^{ex} and fitness $r_t^{fitness}$
10. Store transition $\left(s_t, a_t, r_t^{ex}, r_t^{fitness}, s_{t+1}\right)$ in $\mathcal{D}^{replay}$
11. **End for**
12. **For** *step* $\in \{0, \ldots, LearningSteps\}$ **do**
13. Sample a random mini-batch of n transitions from $\mathcal{D}^{replay}$
14. Calculate loss L_{Critic} and $L_{Fitness}$, and update θ^{Q} and $\theta^{Q^{fitness}}$
15 Calculate $\nabla_{\theta^{in}} \theta^{\mu}$ and $\nabla_{\theta^{\mu}} L_{Actor}$, and update θ^{μ}
16. Calculate $\nabla_{\theta^{\mu}} J^{fitness}(\mu)$ and $\nabla_{\theta^{in}} L_{Intinsic}$, and update θ^{in}
17. Update the target network:

(*continued on next page*)

(*continued*)

Algorithm 1 IRDDPG learning algorithm

$\theta^{Q'} \leftarrow \tau\theta^{Q} + (1-\tau)\theta^{Q'}$
$\theta^{\mu'} \leftarrow \tau\theta^{\mu} + (1-\tau)\theta^{\mu'}$
$\theta^{Q^{fitness}'} \leftarrow \tau\theta^{Q^{fitness}} + (1-\tau)\theta^{Q^{fitness}'}$
End for
End for

5. Experiment and result analysis

We evaluated our approach in the HRC manufacturing scenario that one human and one industrial robot working in a shared workspace. Here the model of ABB industrial robot IRB 1200 7/0.7 was used to simulate in the virtual environment. During the collaboration process, the human arm was the only obstacle to be considered. As discussed in section 3, given the fitness function (Definition 2), the safe HRC manufacturing problem (Problem 1) is to find the optimal policy to maximize the expected accumulated fitness, and the reward designed problem (Problem 2) is to find the optimal reward function to help the agent learn the optimal policy. Therefore, the objective of the experiments is to test whether our approach can help find a better reward function to learn a policy and then improve the expected accumulated fitness.

Table 2
Network architectures parameters.

Networks	Input	Dimensions	Activation function
Fitness network Critic network	(s_t, a_t)	(24, 200, 400, 20, 1)	$(ReLu, ReLu, ReLu, -)$
Intrinsic reward network			$(ReLu, ReLu, ReLu, tanh)$
Actor-network	s_t	(18, 400, 400, 50, 6)	

Table 3
Hyper-parameters used in experiments.

Parameters	Definitions	Values
Δt	Interaction interval	0.05 s
d_{safe}	Safe distance	0.1 m
τ	Soft update factor	0.01
γ	Discount factor	0.9
T	Max time steps	150
$T_{expected}$	The expected task completion time	60
k	Action bound	0.698 rad/s
η_1	Learning rate for policy module	0.0005
η_2	Learning rate for intrinsic reward module	0.0001
ζ	The initial variance of Gaussian noise $\mathcal{N}$	2
n	Numbers of mini-batch	64
M	The capacity of the replay buffer $\mathcal{D}^{replay}$	100,000

At first, the extrinsic reward function needs to be manually designed based on the designer's prior knowledge. Therefore, we designed the extrinsic reward function used throughout the experiment based on the Eq. (3-2) and the fitness function:

$$R^{ex}(s_t, a_t) = \lambda_1 \cdot R^{ex}_{risk}(s_t, a_t) + \lambda_2 \cdot R^{ex}_{task}(s_t, a_t) \tag{13}$$

where the $R^{ex}_{risk}(s_t, a_t)$ and $R^{ex}_{task}(s_t, a_t)$ are defined as equation:

$$R^{ex}_{risk}(s_t, a_t) = \sum_{i=1}^{4} risk\left(\|d_{i,t}\|\right) \tag{14}$$

$$risk\left(\|d_{i,t}\|\right) = \begin{cases} 0 & \|d_{i,t}\| \geq d_{safe} \\ \left(\dfrac{d_{safe}}{d_{safe} + \|d_{i,t}\|}\right)^4 & 0 < \|d_{i,t}\| < d_{safe} \\ -3 & \|d_{i,t}\| \leq 0 \end{cases} \tag{15}$$

$$R^{ex}_{task}(s_t, a_t) = 10 \cdot I\left(\|c_t - c_{target}\| \leq \varepsilon\right) - \frac{\|c_t - c_{target}\|}{\|c_{initial} - c_{target}\|} \tag{16}$$

where λ_1 and λ_2 are weight contribution corresponding to each optimization goal, $I(\cdot)$ is the indicator function, and the extrinsic reward is the weighted sum of $R^{ex}_{risk}(s_t, a_t)$ and $R^{ex}_{task}(s_t, a_t)$. In the following experiment, different allocations of weight contribution were selected to test the impact of extrinsic reward function on policy learning, and the learned policy performance was also compared with the policy learned with introducing the intrinsic reward function.

5.1. Experiment setting and implementation details

As discussed in Section 4.2, the trained agent contains two modules: the intrinsic reward module and the policy module, and each module

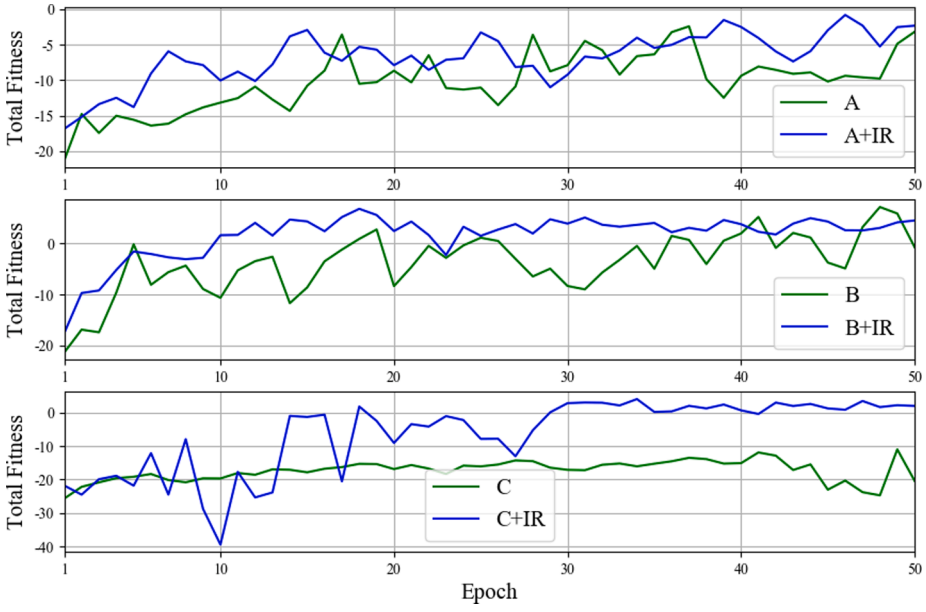


Fig. 7. Learning curve on total fitness of agents.

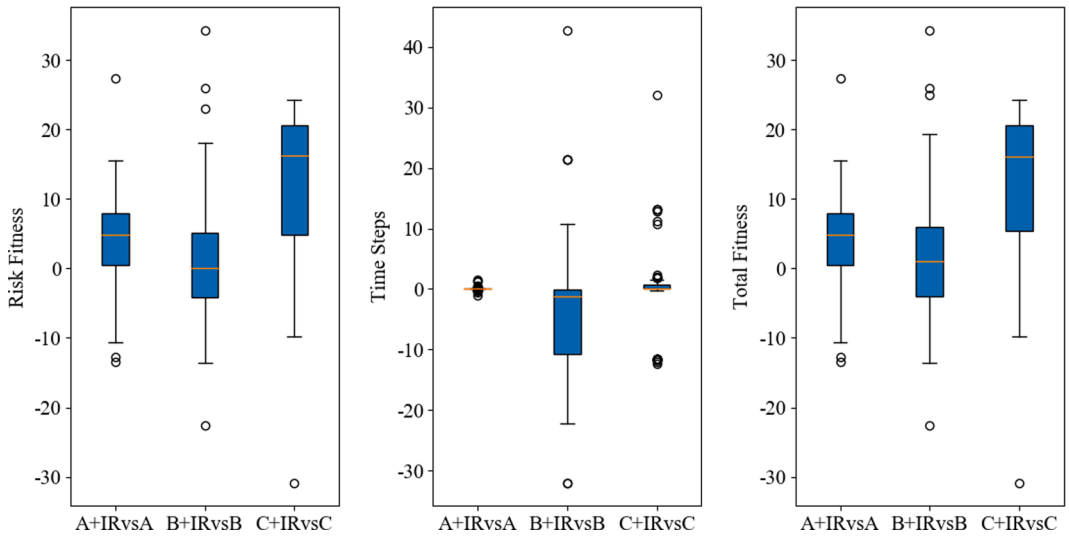


Fig. 8. Performances comparisons between agents.

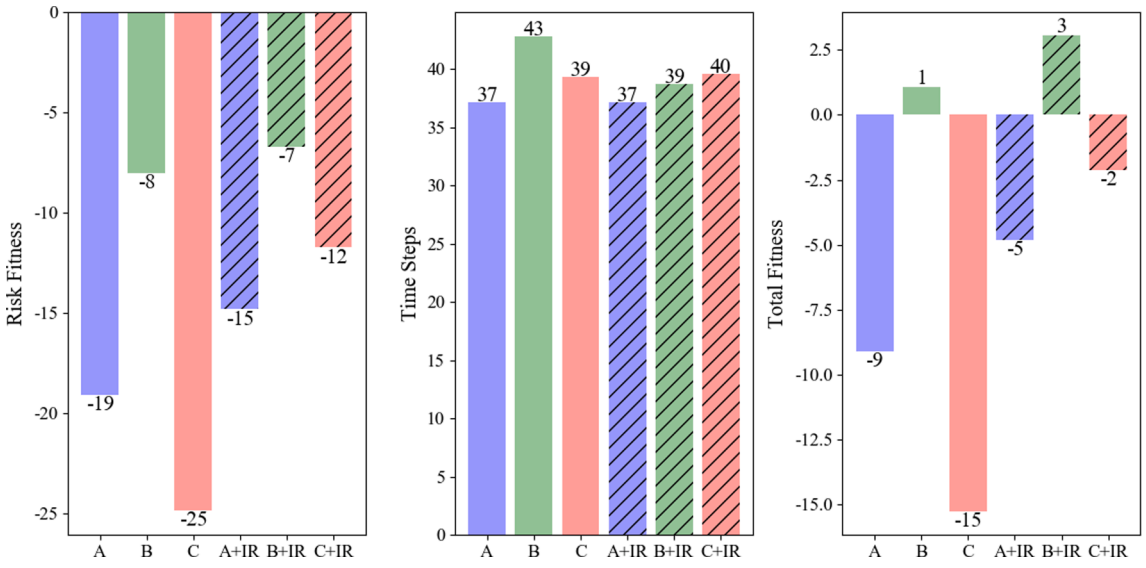


Fig. 9. Average performances of different agents.

Table 4
Average deviations on each criterion.

Agents	δ on risk fitness	δ on time steps	δ on total fitness
Without IR	11.02	7.13	11.25
With IR	7.85	5.57	7.97

Table 5
Average deviations on each criterion.

Algorithms	δ on risk fitness	δ on time steps	δ on total fitness
IRDDPG	−7.0	39.0	3.0
VOCA	−43.1	59.8	−36.1
TD3	−7.7	43.8	0.94

contains two main neural networks. The network architectures parameters of the agent are given in Table 2.

In this experiment, all the networks were built by three hidden layer neural networks, and except for the actor-network, the other networks

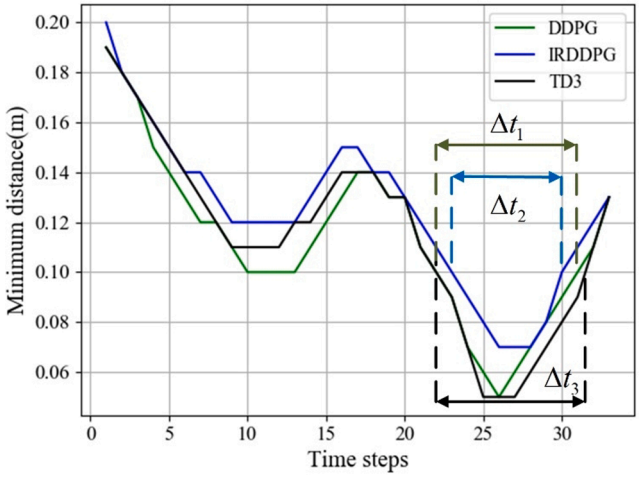


Fig. 10. Minimum distance comparison of DDPG, IRDDPG and TD3 algorithm.

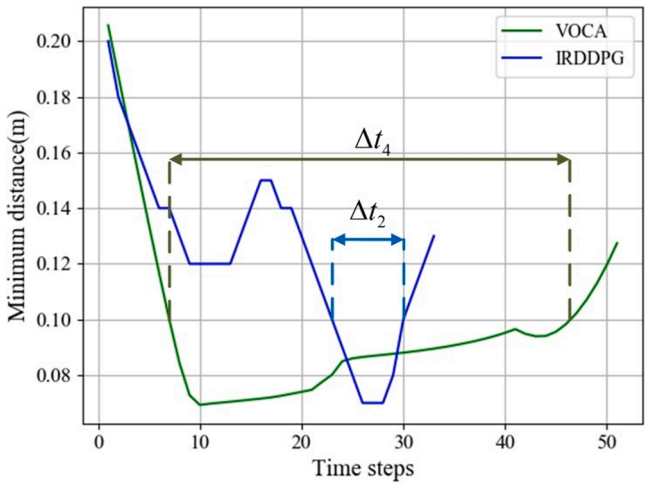


Fig. 11. Minimum distance comparison of IRDDPG and VOCA.

have the same network architectures. All the hidden layers are implemented by fully connected neural networks, and their dimensions are (400, 400, 50) for the actor-network and (200, 400, 20) for the other networks. For the two estimation networks, the *ReLu* activation function is used for hidden layer outputs while the two networks output a single scalar without activation function. The input for the first three networks is the concatenation of system state s_t and action a_t, which is a 24-dimensional vector. For the actor-network, the input is the system state s_t, which is an 18-dimensional vector, and the output is the velocity commands for each joint. For the intrinsic reward network and the actor-network, a *tanh* activation function is applied to keep the two networks output in $[-1, 1]$, and the action bound k is multiplied to the output of the actor-network to make the action in an appropriate range.

To test the impact of weights contribution of extrinsic reward function on policy learning and evaluate our approach, we trained several agents with different settings. The agents' settings are listed below.

- Agent A and agent A with intrinsic reward function (A + IR): $\lambda_1 = 1$, $\lambda_2 = 1$
- Agent B and agent B with intrinsic reward function (B + IR): $\lambda_1 = 2$, $\lambda_2 = 1$
- Agent C and agent C with intrinsic reward function (C + IR): $\lambda_1 = 1$, $\lambda_2 = 2$

According to the weights contributions of the extrinsic reward function, three groups of agents are set. Each group of agents has the same extrinsic reward function. The only difference between the agents in one group is whether there is an intrinsic reward function. Different weights contributions indicate the proportion of each optimization goal to the total fitness. The agents only using extrinsic reward function are named agent A, B and C, respectively, and are trained by the DDPG algorithm. For these three agents, there are three other agents with intrinsic reward function corresponding to them, which were trained by the proposed IRDDPG algorithm. During one interaction for the agent with the simulation environment, the initial task state $c_{initial}$ and the target task state c_{target} are randomly selected from task space C_{space}, and the obstacle is randomly moving at a constant and slow speed in front of the industrial robot. The expected behaviour of the agent is to move from $c_{initial}$ to c_{target} in the expected time and try to evade the obstacle. For all agents, the same hyperparameters are used in this experiment, as shown in Table 3.

5.2. Results and analysis

In the following experiment, the agent was implemented by Tensorflow and trained on an NVIDIA GeForce GTX 1060 GPU. Our experiments contain three phases: training in simulation, comparison and analysis, and evaluation in simulation.

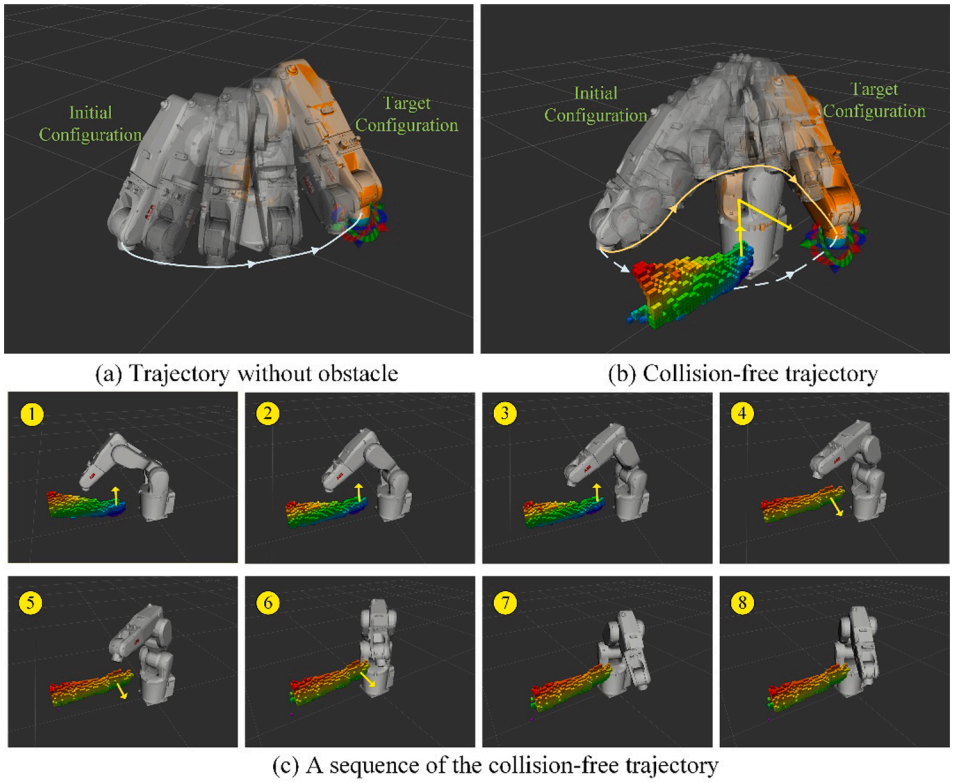


Fig. 12. Collision avoidance on a dynamic obstacle.

(1) Training in simulation

For the phase of training in simulation, because the expected fitness reflects the agents' expected behaviour with different reward functions, we compared the learning curve on total fitness of the agent using intrinsic reward function with the agent only using extrinsic reward function, as shown in Fig. 7. The total fitness was averaged for each epoch which contains 200 episodes, and all agents were trained for 10,000 episodes. From Fig. 7, we can see that the agents with intrinsic reward function get a more stable learning curve than those with only extrinsic reward function. For agent A and agent C, introducing the intrinsic reward function improves the policy performance, and the learning curve of agent C with the intrinsic reward function shows a significant improvement. For agent B, our approach gets a comparable policy performance.

(2) Comparison and analysis

After the training process, we evaluated the agents in two aspects. One is how much our approach improves the performance on each optimization goal, and another is the impact of introducing the intrinsic reward function on the extrinsic reward function and the policy learning. Firstly, we randomly generated 1000 test scenarios, and then the six agents were tested on the scenarios, respectively. Then, for each optimization goal and the total fitness, we compared the differences between agents with different reward functions, as shown in Fig. 8. The risk fitness and total fitness are calculated according to the fitness function, and the time steps are the task completion time. For each criterion, we subtract the value of agents only with the extrinsic reward function from the agents with the intrinsic reward function to get the comparison results on every test scenario. Fig. 8 shows the statistics on the comparison results in box plots. Because the risk fitness is reflected by a negative value, the subtract result on the risk fitness greater than zero reflects that the agent with intrinsic reward function performs better, which is the same on the total fitness. For the time steps, the subtract result less than zero indicates that the IRDDPG agent performs better.

The median of the box plot can intuitively show which agent in the same group performs better on each criterion. From the perspectives of risk fitness and total fitness, the upper quartiles of box plots of the first and third group are both greater than zero, which indicates that agents A and C with intrinsic reward function achieved a better performance in more than 75% of test scenarios. For agent B, the median of the box plot is equal to zero, which indicates that the two agents achieved a comparable performance. Similarly, in terms of completion time steps, agent B with intrinsic reward function got a slight improvement, and agents A and C got a comparable performance. The results indicate that adopting the intrinsic reward function to help the agents to learn policy can effectively improve performance.

To evaluate the impact of introducing the intrinsic reward function on the extrinsic reward function, the average performances of different agents on each criterion were computed over the 1000 test scenarios, as shown in Fig. 9.

As can be seen from Fig. 9, for the two criteria of risk fitness and total fitness, the average performance of the IRDDPG agents is better than that of the DDPG agents, while in terms of the time steps, the two algorithms achieved similar performance. Moreover, we can observe the performance differences of different agents on each criterion from Fig. 8. The average performances of agents A, B and C indicate that different weights contributions on extrinsic reward function greatly affect the performance of the policy. However, introducing the intrinsic reward function reduces the deviation of performances among different policy caused by weights contributions diversity.

To precisely evaluate the deviation of different policy, we calculated the average deviations of agents with intrinsic reward function and agents only with extrinsic reward function, respectively. The average deviation of agents only with extrinsic reward function can be calculated by

$$\delta = \frac{1}{3000}\sum_{i=1}^{1000}\left(\left|x_i^A - \bar{x}\right| + \left|x_i^B - \bar{x}\right| + \left|x_i^C - \bar{x}\right|\right) \tag{17}$$

where $\bar{x} = \frac{1}{3000}\sum_{i=1}^{1000}\left(x_i^A + x_i^B + x_i^C\right)$, and x_i^A, $x_i^B$and $x_i^C$are the performance values on ith test scenario of agent A, agent B and agent C, respectively. The average deviation of agents with intrinsic reward function can be obtained similarly. The average deviations on each criterion are given in Table 4. From Table 4, we can see that the average deviations of agents with intrinsic reward function are less than agents without the intrinsic reward function.

In the above experiments, the IRDDPG agents and DDPG agents are analyzed from three perspectives. Firstly, under the same extrinsic reward function, the performances of the policy of different algorithms are compared. Then, the average performance of each agent on each criterion is compared, and the average deviations of two algorithms on each criterion are calculated and analyzed. Based on the above results, the following conclusions can be obtained:

(1) Introducing the intrinsic reward function can effectively help the agent improve the performance of the policy.
(2) For the extrinsic reward function with different weights contributions, introducing the intrinsic reward function can reduce the differences between different policy due to the deviation of weights contributions, which can reduce the adjustment process of weights.

Then, we compared the proposed IRDDPG algorithm with the velocity-based online collision avoidance (VOCA) [49] algorithm and TD3 [50] algorithm. The average performance of each algorithm on each criterion is calculated and compared, and the value of each criterion is the average in 1000 test scenarios, as shown in Table 5, where the IRDDPG algorithm used agent B with intrinsic reward function for testing, and the TD3 algorithm used the same hyper-parameters.

Table 5 shows the average comparison of the three algorithms on three criteria. It can be seen that the average risk fitness and total fitness of the IRDDPG algorithm are larger than that of the VOCA algorithm and the TD3 algorithm, and the average time steps are shorter. Therefore, the proposed IRDDPG algorithm is safer during one interaction and has higher manufacturing efficiency.

(3) Evaluation in simulation

The experimental results in the comparison and analysis phase show that the agents in the second group achieved a better performance of the policy. Thus we compared the minimum distance during one interaction of the two agents with the VOCA algorithm and TD3 algorithm, as shown in Figs. 10 and 11.

As shown in Figs. 10 and 11, the agents of the four algorithms can successfully avoid obstacles. The time that the minimum distance is less than the safe distance of the proposed IRDDPG algorithm is shorter than the other three algorithms, that is $\Delta t_2 < \Delta t_1 < \Delta t_3 < \Delta t_4$, so the IRDDPG algorithm can be more able to guarantee safety for human. Meanwhile, the task completion time of the IRDDPG algorithm is the same as the DDPG and TD3 algorithm and less than the VOCA algorithm.

Then, we conduct the simulation experiment in ROS to evaluate the trained agent. The collision avoidance on a dynamic obstacle is shown in Fig. 12, where the obstacle is the human arm, and its spatial representation is given in the form of an octree map.

Fig. 12(a) shows the trajectory of the industrial robot without obstacle in the workspace, and Fig. 12(b) shows the collision-free trajectory of the industrial robot when the obstacle moves along the path (the yellow line, the obstacle moves upwards first and then moves horizontally to the right). Fig. 12(c) illustrates the sequence of the

collision-free trajectory. As can be seen from the first three screenshots labelled by 1, 2, 3, respectively, the distance of the industrial robot from the obstacle gradually decreases during the movement to the target configuration. Therefore, the industrial robot needs to adjust its trajectory to avoid a collision. Meanwhile, the obstacle is also moving towards the direction of the robot evading. For screenshots 5, 6 and 7, the obstacle moves horizontally to the right. Although the obstacle movement will not collide with the industrial robot at this time, the industrial robot still takes action to increase the distance.

6. Conclusions

For industrial HRC, the safety issue is the priority during the manufacturing operations. To ensure the safety of human and manufacturing productivity in the unstructured collaborative environment, it is of significant importance for industrial robots to have autonomously learning and active response capabilities. Considering the scenario that one human and one industrial robot perform collaborative manufacturing in a shared workspace, this paper first utilized the capsule bounding box to construct the collision detection model for HRC. On this basis, the safe HRC manufacturing problem was described and formulated into an MDP. Then, aiming at the problem that the design of reward function is difficult and error-prone, this paper gave the mathematical expression of the reward function design problem based on the optimal reward framework and proposed the DPG-based reward function optimizing approach to solving it. The proposed approach introduced a parameterized intrinsic reward function to help the robot to learn the expected policy. Next, the IRDDPG algorithm, which is the combination of the DDPG algorithm and the reward function optimizing approach, is proposed to realize the safety assurance control of the industrial robot. The experimental results show the effectiveness of our algorithm in HRC. To be more specific, compared with the classical algorithm VOCA and DRL algorithms DDPG and TD3, the proposed IRDDPG algorithm achieved a better performance in reducing risk and ensuring manufacturing productivity. The results also indicate that our reward function optimizing approach can improve policy performance and reduce the deviation of performances among different policy caused by weights contributions diversity.

Overall, this work focus on the generalization of the deep reinforcement learning theory into the industrial human-robot collaboration. Concretely, a specifically designed reward mechanism, which is the main contribution of this research, enhances the proposed method. However, this method relays on the perception data being correct, and up to now, it only runs within the laboratory environment. Since deep reinforcement learning and intelligent robotics are still at the cutting-edge of artificial intelligence research, future works may focus on the investigations of higher dimension and amount of data, generalization to more complex scenarios, experiments with real physical robots based on enhanced sensing methods, advanced reward function integrating with smooth control, as well as new theories and algorithms customized for industrial tasks. More powerful hardware and elaborate training simulators are also expected in future works.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

This work is supported by the National Natural Science Foundation of China (Grant No. 51775399 and 51675389), the Fundamental Research Funds for the Central Universities (WUT: 2020III047), and the International Science and Technology Innovation Cooperation Project of Sichuan Province (20GJHZ0039). The authors would like to thank Prof. Patrick Doherty from the Department of Computer and Information Science (IDA) at Linköping University for his support to improve this manuscript.

References

[1] Q. Liu, Z. Liu, W. Xu, et al., Human-robot collaboration in disassembly for sustainable manufacturing, Int. J. Prod. Res. 57 (12) (2019) 4027–4044.
[2] R.J. Halme, M. Lanz, J. Kmrinen, et al., Review of vision-based safety systems for human-robot collaboration, Procedia CIRP 72 (1) (2018) 111–116.
[3] S. Robla-Gomez, V.M. Becerra, J.R. Llata, et al., Working together: a review on safe human-robot collaboration in industrial environments, IEEE Access 5 (2017) 26754–26773.
[4] K. Wu, M.A. Esfahani, S. Yuan, et al., Depth-based obstacle avoidance through deep reinforcement learning, in: International Conference on Mechatronics and Robotics Engineering, 2019, pp. 102–106.
[5] D. Hadfieldmenell, S. Milli, P. Abbeel, et al., Inverse reward design, in: Neural Information Processing Systems, 2017, pp. 6765–6774.
[6] D. Amodei, C. Olah, J. Steinhardt, et al., Concrete Problems in AI Safety. arXiv: cs. AI, 2016, 1606.06565.
[7] D. Silver, G. Lever, N. Heess, et al., Deterministic policy gradient algorithms, in: International Conference on Machine Learning, 2014, pp. 387–395.
[8] T.P. Lillicrap, J.J. Hunt, A. Pritzel, et al., Continuous control with deep reinforcement learning. International Conference on Learning Representations, 2016.
[9] V. Villani, F. Pini, F. Leali, et al., Survey on human-robot collaboration in industrial settings: Safety, intuitive interfaces and applications, Mechatronics 55 (2018) 248–266.
[10] British Standards Institution, ISO 10218-1. Robots and robotic devices-safety requirements for industrial robots-Part 1: Robots, 2011.
[11] British Standards Institution, ISO/TS 15066. Robots and robotic devices-collaborative robots, 2016.
[12] J.A. Marvel, R. Norcross, Implementing speed and separation monitoring in collaborative robot workcells, Rob. Comput. Integr. Manuf. 44 (2017) 144–155.
[13] Z. Liu, X. Wang, Y. Cai, et al., Dynamic risk assessment and active response strategy for industrial human-robot collaboration, Comput. Ind. Eng. 141 (2020), 106302.
[14] W.P. Shackleford, G.S. Cheok, T. Hong, et al., Performance Evaluation of Human Detection Systems for Robot Safety, J. Intell. Rob. Syst. 83 (1) (2016) 85–103.
[15] F. Flacco, T. Kroeger, A. De Luca, et al., A Depth Space Approach for Evaluating Distance to Objects, J. Intell. Rob. Syst. 80 (1) (2015) 7–22.
[16] A. Mohammed, B. Schmidt, L. Wang, Active collision avoidance for human-robot collaboration driven by vision sensors, Int. J. Comput. Integr. Manuf. 30 (9) (2017) 970–980.
[17] M. Ragaglia, A.M. Zanchettin, P. Rocco, Trajectory generation algorithm for safe human-robot collaboration based on multiple depth sensor measurements, Mechatronics 55 (2018) 267–281.
[18] J.A. Marvel, J. Falco, I. Marstio, Characterizing task-based human-robot collaboration safety in manufacturing, IEEE Trans. Syst. Man Cybern. Syst. 45 (2) (2015) 260–275.
[19] F. Vicentini, M. Giussani, L.M. Tosatti, Trajectory-dependent safe distances in human-robot interaction, in: IEEE Emerging Technology and Factory Automation, 2014, pp. 1–4.
[20] M.P. Polverini, A.M. Zanchettin, P. Rocco, Real-time collision avoidance in human-robot interaction based on kinetostatic safety field, in: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2014, pp. 4136–4141.
[21] A.M. Zanchettin, N.M. Ceriani, P. Rocco, et al., Safety in human-robot collaborative manufacturing environments: metrics and control, IEEE Trans. Autom. Sci. Eng. 13 (2) (2016) 882–893.
[22] M. Rubagotti, T. Taunyazov, B. Omarali, et al., Semi-autonomous robot teleoperation with obstacle avoidance via model predictive control, IEEE Rob. Autom. Lett. 4 (3) (2019) 2746–2753.
[23] M.G. Mohanan, A. Salgoankar, A survey of robotic motion planning in dynamic environments, Rob. Auton. Syst. 100 (2017) 171–185.
[24] S. Lo, C. Cheng, H. Huang, Virtual Impedance Control for Safe Human-Robot Interaction, J. Intell. Rob. Syst. 82 (1) (2016) 3–19.
[25] B. Schmidt, L. Wang, Contact-less and programming-less human-robot collaboration, Procedia CIRP 7 (2013) 545–550.
[26] I. Kardan, A. Akbarzadeh, A.M. Mohammadi, Real-time velocity scaling and obstacle avoidance for industrial robots using fuzzy dynamic movement primitives and virtual impedances, Ind. Robot 45 (1) (2018) 110–126.
[27] S. Gu, E. Holly, T. Lillicrap, et al., Deep reinforcement learning for robotic manipulation with asynchronous off-policy updates, in: IEEE International Conference on Robotics and Automation, 2017, pp. 3389–3396.
[28] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.
[29] V. Mnih, K. Kavukcuoglu, D. Silver, et al., Human-level control through deep reinforcement learning, Nature 518 (7540) (2015) 529–533.
[30] F. Zhang, J. Leitner, M. Milford, et al., Towards vision-based deep reinforcement learning for robotic motion control, in: IEEE International Conference on Robotics and Automation, 2015, pp. 1884–2020.
[31] B. Sangiovanni, G.P. Incremona, M. Piastra, et al., Self-configuring robot path planning with obstacle avoidance via deep reinforcement learning, IEEE Control Syst. Lett. 5 (2) (2021) 397–402.

[32] S. Gu, T. Lillicrap, I. Sutskever, et al., Continuous deep Q-learning with model-based acceleration, in: International Conference on Machine Learning, 2016, pp. 2829–2838.
[33] B. Sangiovanni, A. Rendiniello, G.P. Incremona, et al., Deep reinforcement learning for collision avoidance of robotic manipulators, in: European Control Conference, 2018, pp. 2063–2068.
[34] S. Paul, L. Vig, Deterministic Policy gradient based robotic path planning with continuous action spaces, in: IEEE International Conference on Computer Vision Workshops, 2017, pp. 725–733.
[35] C. Do, C. Gordillo, W. Burgard, Learning to pour using deep deterministic policy gradients, in: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2018, pp. 3074–3079.
[36] A.Y. Ng, D. Harada, S. Russell, Policy invariance under reward transformations: theory and application to reward shaping, in: International Conference on Machine Learning, 1999, pp. 278–287.
[37] H. Tang, R. Houthooft, D. Foote, et al., #Exploration: A study of count-based exploration for deep reinforcement learning, in: Neural Information Processing Systems, 2017, pp. 2750–2759.
[38] G. Ostrovski, M.G. Bellemare, A.V. Den Oord, et al., Count-Based exploration with neural density models, in: International Conference on Machine Learning, 2017, pp. 2721–2730.
[39] D. Pathak, P. Agrawal, A.A. Efros, et al., Curiosity-driven Exploration by Self-supervised Prediction, in: International Conference on Machine Learning, 2017, pp. 2778–2787.
[40] R. Rahmatizadeh, P. Abolghasemi, A. Behal, et al., From virtual demonstration to real-world manipulation using LSTM and MDN, in: National Conference on Artificial Intelligence, 2018, pp. 6524–6531.
[41] A.Y. Ng, S. Russell, Algorithms for Inverse Reinforcement Learning, International Conference on Machine Learning (2000) 663–670.
[42] M. Pfeiffer, M. Schaeuble, J. Nieto, et al., From perception to decision: A data-driven approach to end-to-end motion planning for autonomous ground robots, in: International Conference on Robotics and Automation, 2017, pp. 1527–1533.
[43] S. Singh, R.L. Lewis, A.G. Barto, Where Do Rewards Come From?, in: Annual Conference of the Cognitive Science Society, 2009, pp. 2601–2606.
[44] S. Singh, R.L. Lewis, A.G. Barto, et al., Intrinsically Motivated Reinforcement Learning: An Evolutionary Perspective, IEEE Trans. Auton. Ment. Dev. 2 (2) (2010) 70–82.
[45] J. Sorg, R.L. Lewis, S. Singh, Reward Design via Online Gradient Ascent, in: Neural Information Processing Systems, 2010, pp. 2190–2198.
[46] Z. Zheng, J. Oh, S. Singh, On Learning Intrinsic Rewards for Policy Gradient Methods, in: Neural Information Processing Systems, 2018, pp. 4644–4654.
[47] R.S. Sutton, A.G. Barto, Reinforcement Learning: An Introduction, MIT Press, 2018.
[48] Y. Choi, J. Chang, W. Wang, et al., Continuous collision detection for ellipsoids, IEEE Trans. Visual Comput. Graphics 15 (2) (2009) 311–324.
[49] H.C. Lin, Y. Fan, T. Tang, et al., Human guidance programming on a 6-DoF robot with collision avoidance, in: IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS), 2016, pp. 2676–2681.
[50] S. Fujimoto, H. Van Hoof, D. Meger, Addressing Function Approximation Error in Actor-Critic Methods, in: International Conference on Machine Learning, 2018, pp. 1587–1596.